

SQL Cheat Sheet

29 tables · 292 concepts · Updated 2026-09-27 · cheatgrid.com

SQL (Structured Query Language) is the standard language for retrieving, analyzing and changing data held in relational databases. It is used across virtually every industry, from querying a single table to orchestrating multi-table joins with aggregations and window functions. SQL is **declarative**: you specify *what* you want, not *how* to get it, and the database engine handles optimization. The part that trips people up is the **logical execution order** (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY), which differs from the order you write a query in and explains many common pitfalls.

CONTENTS

Basic Query Structure	2
Query Execution Order	3
Operators and Predicates	3
Join Types	4
Aggregate Functions	5
NULL Handling	6
Conditional Logic	7
Advanced Ordering	7
Subqueries & CTEs	8
Set Operations	9
Window Functions: Ranking	10
Window Functions: Aggregate & Analytical	10
Window Frame Specification	11
Advanced Grouping and Pivoting	12
String Functions	13
Pattern Matching	14
Date & Time Functions	15
Numeric Functions	16
Data Types	17
Data Type Conversion	17
DDL Statements	18
Constraints & Keys	19
DML Statements	20
Indexes	21
Views	22
Transactions	23
Access Control	24
Error Handling	25
JSON Functions	26
References	27

TABLE 1

Basic Query Structure

Every SQL query is built from the same six core clauses, but they don't run in the order you write them - the engine processes **FROM** first and **SELECT** near the end, which is why aliases work in **ORDER BY** but not in **WHERE**. Knowing this logical processing order is the single most useful mental model for explaining why **HAVING** filters groups while **WHERE** filters rows, and why row-limiting clauses without **ORDER BY** return non-deterministic results.

Clause	Example	Description
SELECT	<code>SELECT name, salary FROM employees</code>	- Specifies which columns to return and computes their expressions - logically executed near the end (after FROM, WHERE, GROUP BY, HAVING), so its aliases are not visible to those clauses.
SELECT * vs explicit column list	<code>SELECT * FROM employees;</code> <code>SELECT name, salary FROM employees;</code>	* emits every column the table expression produces, in table order - an explicit list names only the columns wanted, so the result doesn't shift when the table gains new ones
FROM	<code>FROM employees</code>	- Identifies the source tables, views, or subqueries and applies any JOINS - logically the first clause processed, so its tables and columns are visible to every later clause.
Column and table aliases (AS)	<code>SELECT e.name AS emp_name FROM employees AS e</code>	- Gives a table or column a temporary name for the rest of the query AS is optional noise, but required when joining a table to itself (self-join)
VALUES lists as a table	<code>SELECT * FROM (VALUES (1,'one'), (2,'two')) AS t(num, letter);</code>	- Builds a constant table from literal rows without creating one on disk - can appear anywhere a SELECT can, most often as the row source for INSERT
WHERE	<code>WHERE salary > 50000</code>	- Filters individual rows before grouping - cannot reference aggregate functions or SELECT-list aliases (SELECT hasn't run yet).
GROUP BY	<code>GROUP BY department</code>	- Collapses rows into one row per distinct group so aggregates like SUM and COUNT can be applied - every non-aggregated column in SELECT must also appear here.
HAVING	<code>HAVING COUNT(*) > 10</code>	- Filters groups after aggregation - the only clause where aggregate functions are valid filter conditions.
ORDER BY	<code>ORDER BY salary DESC</code>	- Sorts the final result set - runs after SELECT, so it can reference SELECT-list aliases; ASC is the default direction.
DISTINCT	<code>SELECT DISTINCT country FROM customers</code>	- Removes duplicate rows from the result, comparing the full tuple of selected columns - not per-column: every selected column counts toward uniqueness.
LIMIT / TOP / FETCH FIRST	<code>LIMIT 10</code> <code>TOP 10</code> <code>FETCH FIRST 10 ROWS ONLY</code>	- Caps the number of rows returned - syntax varies by dialect: LIMIT in PostgreSQL, MySQL, SQLite, Snowflake; TOP in SQL Server; FETCH FIRST is the ANSI standard supported by Oracle, DB2, Snowflake, and PostgreSQL.
OFFSET	<code>LIMIT 10 OFFSET 20</code>	Skips a number of rows before returning results, typically for pagination - without ORDER BY the skipped subset is undefined, so different pages can repeat or miss rows.

TABLE 2

Query Execution Order

The order you write SQL clauses in is not the order the database evaluates them. This logical processing order explains most "why doesn't this work" errors: why **WHERE** cannot see **SELECT** aliases, why **GROUP BY** rejects non-aggregated columns, and why window functions cannot be filtered in **WHERE**.

Step	Example	Description
1. FROM / JOIN	<code>FROM orders o JOIN customers c ON o.customer_id = c.id</code>	• First step: identifies source tables and performs joins • table aliases defined here are available to every later clause.
2. WHERE	<code>WHERE o.status = 'shipped'</code>	• Filters individual rows before any grouping • cannot reference SELECT aliases, aggregate functions, or window functions.
3. GROUP BY	<code>GROUP BY c.country</code>	Collapses rows into groups for aggregate computation: every non-aggregated column in SELECT must appear here.
4. HAVING	<code>HAVING COUNT(o.id) > 5</code>	• Filters groups after aggregation • can reference aggregate functions; cannot reference SELECT aliases in standard SQL.
5. SELECT (+ window functions)	<code>SELECT c.country, COUNT(o.id), SUM(o.amount)</code>	Evaluates expressions, aliases, and window functions: aliases defined here are why WHERE / GROUP BY / HAVING can't see them.
6. QUALIFY	<code>QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) = 1</code>	• Filters on window function results: what HAVING is to GROUP BY, QUALIFY is to window functions • Snowflake, BigQuery, DuckDB, Databricks; not ANSI standard.
7. DISTINCT	<code>SELECT DISTINCT country</code>	Removes duplicate rows from the result set after SELECT expressions are evaluated.
8. ORDER BY	<code>ORDER BY SUM(o.amount) DESC</code>	• Sorts the final result set: runs after SELECT, so it CAN reference SELECT aliases • without it, row order is not guaranteed.
9. LIMIT / OFFSET / TOP / FETCH	<code>LIMIT 10 OFFSET 20</code>	• Restricts and paginates the already-sorted result • without ORDER BY the rows returned are unpredictable.

TABLE 3

Operators and Predicates

Comparison and logical operators combine and test values in **WHERE** and **ON** clauses; three-valued logic means any comparison against **NULL** yields unknown rather than true or false.

Concept	Example	Description
Comparison operators	<code>WHERE price >= 100</code>	• <code>=</code>, <code><></code> (or <code>!=</code>), <code><</code>, <code>></code>, <code><=</code>, <code>>=</code> test two values • null on either side yields unknown, not true or false.
AND / OR / NOT	<code>WHERE status = 'active' AND (region = 'EU' OR region = 'UK')</code>	• Combines conditions with three-valued logic • NOT binds tighter than AND, which binds tighter than OR, so mixed conditions need parentheses.
IN	<code>WHERE country IN ('US', 'CA', 'MX')</code>	• True if the value equals any item in the list • shorthand for chained OR comparisons
NOT IN (NULL trap)	<code>WHERE id NOT IN (SELECT id FROM excluded)</code>	• True only if the value is unequal to every item in the list • returns null (no rows), not true, if any list item is null
BETWEEN ... AND	<code>WHERE age BETWEEN 18 AND 65</code>	• Range test inclusive of both endpoints • equivalent to <code>age >= 18 AND age <= 65</code>.
IS DISTINCT FROM	<code>WHERE a IS DISTINCT FROM b</code>	• Compares like <code><></code> but treats NULL as a normal, comparable value • never returns null, unlike a plain <code><></code>

Concept	Example	Description
IS NOT DISTINCT FROM	<code>WHERE a IS NOT DISTINCT FROM b</code>	Compares like <code>=</code> but treats two nulls as equal instead of unknown.
ANY / SOME	<code>WHERE price > ANY (SELECT price FROM budget)</code>	- True if the comparison holds for at least one row of the subquery SOME is a synonym, and IN is equivalent to <code>= ANY</code>.
ALL	<code>WHERE price > ALL (SELECT price FROM budget)</code>	True only if the comparison holds for every row the subquery returns (also true when it returns none).
Row value comparison	<code>WHERE (last_name, first_name) < ('Smith', 'Ann')</code>	Compares two row constructors field by field, left to right, stopping at the first pair that differs or is null.

TABLE 4

Join Types

Joins combine rows from two or more tables into a single result. Choosing the right join controls which rows survive when keys don't match on every side, and a misplaced filter or wrong join type is one of the most common sources of silently wrong query results.

Type	Example	Description
INNER JOIN	<code>SELECT * FROM orders o INNER JOIN customers c ON o.customer_id = c.id</code>	- Returns only rows with matching values in both tables - the default and most common join - rows with NULL keys on either side are excluded because NULL never equals NULL
LEFT JOIN (LEFT OUTER JOIN)	<code>SELECT * FROM customers c LEFT JOIN orders o ON c.id = o.customer_id</code>	- Returns all rows from the left table plus matching right rows, NULL when there's no match - a filter on a right-table column in <code>WHERE</code> silently turns the join back into an INNER JOIN, so keep such filters in <code>ON</code>
RIGHT JOIN (RIGHT OUTER JOIN)	<code>SELECT * FROM orders o RIGHT JOIN customers c ON o.customer_id = c.id</code>	- Returns all rows from the right table plus matching rows from the left - mirror of LEFT JOIN; rarely used because flipping the table order to a LEFT JOIN is more readable and more portable
FULL OUTER JOIN	<code>SELECT * FROM customers c FULL OUTER JOIN orders o ON c.id = o.customer_id</code>	- Returns all rows from both tables, with NULLs for the side that didn't match - not supported in MySQL: emulate with a <code>UNION</code> of a LEFT JOIN and a RIGHT JOIN
JOIN ... USING (col)	<code>SELECT * FROM employees e JOIN departments d USING (department_id)</code>	- Shorthand for <code>ON</code> when both tables use the same column name for the join key - the shared column appears only once in the result, unlike a plain <code>ON</code> join
CROSS JOIN	<code>SELECT * FROM colors CROSS JOIN sizes</code>	- Produces a Cartesian product: every row from the first table paired with every row from the second ($N \times M$ rows) - takes no <code>ON</code> condition - the old comma syntax <code>FROM a, b</code> produces the same result
SELF JOIN	<code>SELECT e.name, m.name AS manager FROM employees e JOIN employees m ON e.manager_id = m.id</code>	- Joins a table to itself using two different aliases - useful for hierarchical data (employee to manager) or comparing rows within the same table - aliases are mandatory: without them the column references are ambiguous
Semi-Join / Anti-Join (EXISTS)	<code>-- semi-join: customers who have orders SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id); -- anti-join: customers who have none SELECT * FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);</code>	EXISTS keeps left rows that have a match without duplicating them per matching right row, unlike an INNER JOIN NOT EXISTS keeps left rows that have no match, the standard way to write "rows in A not in B"

Type	Example	Description
LATERAL JOIN (CROSS JOIN LATERAL)	<pre>SELECT d.name, e.name, e.salary FROM departments d CROSS JOIN LATERAL (SELECT name, salary FROM employees WHERE department_id = d.id ORDER BY salary DESC LIMIT 3) e</pre>	- Lets the right-hand subquery reference columns from earlier tables in the FROM clause, evaluated once per outer row, enabling top-N-per-group queries - pair with <code>LEFT JOIN LATERAL ... ON true</code> to keep outer rows that produce no inner rows
CROSS APPLY / OUTER APPLY	<pre>SELECT d.name, e.name FROM departments d CROSS APPLY (SELECT TOP 3 name FROM employees WHERE department_id = d.id ORDER BY salary DESC) e</pre>	- SQL Server's equivalent of LATERAL: evaluates the right-hand table expression once per left-hand row CROSS APPLY drops left rows whose right expression returns no rows (like INNER JOIN) OUTER APPLY keeps them with NULLs on the right (like LEFT JOIN)
NATURAL JOIN	<pre>SELECT * FROM employees NATURAL JOIN departments</pre>	Automatically joins on every column with the same name in both tables, collapsing them into one - risky in production: a later same-named column added to either table silently changes the join condition, so prefer explicit <code>ON</code> or <code>USING (col)</code>

TABLE 5

Aggregate Functions

Aggregate functions collapse many rows into a single summary value over a group, and almost all of them silently ignore NULL inputs. Two traps catch even experienced users: `COUNT(*)` counts every row while `COUNT(col)` skips NULLs in `col`, and `SUM` / `AVG` / `MIN` / `MAX` over an all-NULL or empty group return NULL, never zero.

Function	Example	Description
COUNT()	<pre>SELECT COUNT(*) FROM orders SELECT COUNT(DISTINCT customer_id) FROM orders</pre>	<code>COUNT(*)</code> counts all rows including NULLs <code>COUNT(col)</code> skips NULLs; <code>DISTINCT</code> counts unique values.
SUM()	<pre>SELECT SUM(amount) FROM payments</pre>	Totals all non-NULL values in a column - returns NULL if all values are NULL.
AVG()	<pre>SELECT AVG(salary) FROM employees WHERE dept = 'Sales'</pre>	Arithmetic mean of non-NULL values - NULLs are excluded, not treated as zero.
MAX()	<pre>SELECT MAX(order_date) FROM orders</pre>	- Returns the largest value in a column - works on dates and strings too.
MIN()	<pre>SELECT MIN(price) FROM products WHERE active = 1</pre>	Returns the smallest value in a column.
GROUP_CONCAT / STRING_AGG	<pre>STRING_AGG(name, ', ') WITHIN GROUP (ORDER BY name)</pre>	Concatenates string values from a group into a single delimited string - SQL Server/PostgreSQL use <code>STRING_AGG</code> - MySQL uses <code>GROUP_CONCAT</code>.
FILTER (WHERE ...)	<pre>SELECT count(*) AS total, count(*) FILTER (WHERE i < 5) AS small FROM generate_series(1,10) AS s(i)</pre>	- Feeds only rows matching the condition to one aggregate - avoids a separate <code>CASE</code> per aggregate; PostgreSQL and standard SQL, not T-SQL.
PERCENTILE_CONT / PERCENTILE_DISC	<pre>SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY income) FROM households</pre>	<code>PERCENTILE_CONT</code> interpolates between adjacent values (continuous) <code>PERCENTILE_DISC</code> returns an actual value from the set (discrete).
BOOL_AND / BOOL_OR	<pre>SELECT bool_and(active), bool_or(active) FROM sessions</pre>	<code>BOOL_AND</code> is true only if every non-null input is true <code>BOOL_OR</code> is true if any non-null input is true.

Function	Example	Description
VARIANCE / VAR_SAMP	<pre>SELECT VAR(salary) FROM employees SELECT VARIANCE(salary) FROM employees</pre>	• Statistical variance of the values in a group • VAR/VAR_SAMP (sample), VARP/VAR_POP (population) • name varies by database.
STDDEV / STDEV	<pre>SELECT STDEV(salary) FROM employees</pre>	• Standard deviation of values • STDEV/STDDEV_SAMP (sample), STDEVP/STDDEV_POP (population) • name varies by database.
ANY_VALUE()	<pre>SELECT dept, ANY_VALUE(dept_name) FROM employees GROUP BY dept</pre>	Returns an unspecified, arbitrary non-NULL value from the group, useful for a column that is functionally dependent on the grouping key.
APPROX_COUNT_DISTINCT()	<pre>SELECT APPROX_COUNT_DISTINCT(O_OrderKey) FROM dbo.Orders</pre>	• Estimates <code>COUNT(DISTINCT ...)</code> within about 2% error • far cheaper than an exact distinct count over huge tables.
PRODUCT()	<pre>SELECT PRODUCT(UnitPrice) AS ProductOfPrices FROM Purchasing.PurchaseOrderDetail GROUP BY ProductId</pre>	• Multiplies all non-NULL numeric values in a group • added in SQL Server 2025.

TABLE 6

NULL Handling

SQL treats NULL as "unknown" under three-valued logic, so most operators that touch a NULL return UNKNOWN rather than TRUE or FALSE, which is why `WHERE col = NULL` never matches anything and why `NOT IN` against a subquery containing a NULL silently returns zero rows. The functions and predicates below are the safe, portable tools for testing, substituting, and aggregating NULL data across every major dialect.

Technique	Example	Description
IS NULL / IS NOT NULL	<pre>WHERE phone IS NULL WHERE email IS NOT NULL</pre>	• Tests for NULL values • the only safe way, since <code>= NULL</code> and <code>!= NULL</code> always evaluate to UNKNOWN.
Three-Valued Logic (UNKNOWN)	<pre>SELECT true AND NULL; -- NULL SELECT true OR NULL; -- true</pre>	• Boolean expressions evaluate to TRUE, FALSE, or UNKNOWN (a NULL comparison result) • <code>WHERE</code> keeps only rows where the condition is TRUE, so both FALSE and UNKNOWN rows are dropped.
COALESCE()	<pre>COALESCE(mobile, home_phone, 'N/A')</pre>	• Returns the first non-NULL argument (NULL only if all are NULL) • ANSI standard • accepts any number of arguments • short-circuits left-to-right.
NULLIF()	<pre>NULLIF(denominator, 0)</pre>	• Returns NULL if both arguments are equal, otherwise the first argument • classic use prevents division-by-zero: <code>value / NULLIF(denom, 0)</code>.
ISNULL() / IFNULL() / NVL()	<pre>ISNULL(discount, 0)</pre>	• Two-argument NULL substitution (vendor-specific) • SQL Server <code>ISNULL</code>, MySQL <code>IFNULL</code>, Oracle <code>NVL</code> • less portable than COALESCE.
NULL in aggregates	<pre>AVG(score) (NULL ignored) COUNT(*) (NULL included)</pre>	• Most aggregates skip NULLs in both numerator and denominator • <code>COUNT(*)</code> counts all rows; <code>COUNT(col)</code> skips NULLs.
NULL in JOINS	<pre>a LEFT JOIN b ON a.id = b.id WHERE b.id IS NULL -- anti-join</pre>	• NULL keys never match in join conditions • filtering an outer-join column in <code>WHERE</code> quietly converts LEFT JOIN to INNER JOIN • <code>NOT IN (subquery)</code> returns zero rows if the subquery yields any NULL, so use <code>NOT EXISTS</code> instead.

TABLE 7

Conditional Logic

Conditional expressions let a query branch row-by-row inside `SELECT` , `WHERE` , `ORDER BY` , and `UPDATE` clauses without leaving SQL. `CASE` is the portable, ANSI-standard primitive; vendors layer terser shorthands like `IIF` , `CHOOSE` , and Oracle's `DECODE` on top, each with its own NULL-handling quirks worth knowing.

Expression	Example	Description
CASE WHEN ... THEN ... END	<pre>CASE WHEN salary > 100000 THEN 'Senior' WHEN salary > 60000 THEN 'Mid' ELSE 'Junior' END</pre>	• SQL's primary conditional expression • evaluates WHEN conditions top-down and returns the first match: later conditions are skipped • returns <code>NULL</code> if no WHEN matches and <code>ELSE</code> is omitted.
Simple CASE	<pre>CASE status WHEN 'A' THEN 'Active' WHEN 'I' THEN 'Inactive' ELSE 'Unknown' END</pre>	• Equality shorthand, compares one expression to a list of values using <code>=</code> • cannot match <code>NULL</code> (because <code>NULL = NULL</code> is UNKNOWN), use searched <code>CASE WHEN ... IS NULL</code> for that.
GREATEST() / LEAST()	<pre>GREATEST(price, min_price) LEAST(qty, max_allowed)</pre>	• Returns the largest or smallest value from a list of expressions • NULL arguments are ignored; the result is NULL only if every argument is NULL (in PostgreSQL and MySQL; the SQL standard says NULL wins if any argument is NULL).
IIF()	<pre>IIF(salary > 50000, 'High', 'Low')</pre>	• Ternary shorthand for a two-branch CASE (SQL Server / Access) • rewritten internally as <code>CASE</code> ; nesting is capped at 10 levels, same as CASE.
CHOOSE()	<pre>CHOOSE(quarter, 'Q1', 'Q2', 'Q3', 'Q4')</pre>	• Returns the Nth item from a value list using a 1-based integer index • SQL Server only; an out-of-range or <code>NULL</code> index returns <code>NULL</code> (no error).
DECODE()	<pre>DECODE(status, 1, 'Active', 0, 'Inactive', 'Unknown')</pre>	• Oracle-proprietary equality-based CASE substitute: <code>DECODE(expr, search, result [, ...] [, default])</code> • uniquely treats two <code>NULL</code> s as equal, the opposite of standard <code>CASE</code> .

TABLE 8

Advanced Ordering

Beyond a single `ORDER BY column ASC` , real queries often need per-column sort directions, explicit NULL placement, custom priority orders, locale-aware comparison, or row counts that include ties. The exact syntax and defaults vary across SQL dialects, so portable code typically spells out direction, NULL handling, and tie-handling explicitly rather than relying on the database default.

Technique	Example	Description
ORDER BY ASC / DESC	<pre>ORDER BY salary DESC, name ASC</pre>	• Multi-column sort: secondary columns resolve ties from the primary column • ASC is the default; direction is per-column, so <code>ORDER BY a, b DESC</code> sorts a ASC then b DESC.
NULLS FIRST / NULLS LAST	<pre>ORDER BY score DESC NULLS LAST</pre>	• Explicit NULL positioning in the sort (ANSI standard) • defaults differ: PostgreSQL/Oracle treat NULL as larger, SQL Server/MySQL/SQLite treat it as smaller • SQL Server lacks the keyword; emulate with a <code>CASE WHEN col IS NULL</code> sort key.
ORDER BY column position	<pre>ORDER BY 1, 2</pre>	• Reference by SELECT-list position: 1 = first selected column • convenient for ad-hoc queries but fragile in production: reordering the SELECT list silently changes the sort.
CASE in ORDER BY	<pre>ORDER BY CASE status WHEN 'urgent' THEN 1 WHEN 'normal' THEN 2 ELSE 3 END</pre>	• Custom sort priority using a CASE expression as the sort key • enables business-rule ordering that doesn't follow alphabetical or numeric order.

Technique	Example	Description
COLLATE	ORDER BY name COLLATE Latin1_General_CI_AI	• Overrides the column's collation for that one sort: CI = case-insensitive, AI = accent-insensitive • useful when the column's default collation doesn't match the locale-aware sort you need.
TOP WITH TIES	SELECT TOP 3 WITH TIES name, score FROM leaderboard ORDER BY score DESC	• Includes rows tied with the last qualifying row's ORDER BY value, so the result may exceed N • requires ORDER BY ; SQL Server syntax, equivalent to the standard <code>FETCH FIRST n ROWS WITH TIES</code> .
FETCH FIRST ... WITH TIES	SELECT name, score FROM leaderboard ORDER BY score DESC FETCH FIRST 3 ROWS WITH TIES	• Standard SQL form of <code>TOP WITH TIES</code> : returns extra rows tied for the last place • requires ORDER BY ; supported by PostgreSQL, Oracle, DB2 and SQL Server.

TABLE 9

Subqueries & CTEs

Subqueries nest one query inside another to compute filters, derived tables, or single values, while Common Table Expressions (CTEs) name those intermediate results to break a complex query into readable, sometimes recursive, steps. Knowing when each one short-circuits, gets correlated per row, or is materialized once is the difference between a query that scales and one that crawls.

Technique	Example	Description
Subquery (WHERE)	SELECT * FROM employees WHERE dept_id IN (SELECT id FROM departments WHERE budget > 100000)	• Embeds a query inside another query for filtering • <code>IN</code> acts as a semi-join, matching at most once per outer row even when the subquery returns duplicate rows • the inner query runs once (non-correlated) or per row (correlated).
EXISTS	SELECT name FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id)	• Tests for row existence and stops at the first match • <code>NOT EXISTS</code> is also the safe replacement for <code>NOT IN</code> when the subquery may contain NULLs.
Scalar Subquery	SELECT name, (SELECT MAX(salary) FROM employees) AS max_sal FROM employees	• A subquery used as a single value (one row, one column) in <code>SELECT</code> , <code>WHERE</code> , etc. • returns <code>NULL</code> if zero rows and raises a cardinality error if it returns more than one.
Correlated Subquery	SELECT * FROM employees e1 WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.dept = e1.dept)	• References an outer query's columns inside the subquery • logically executes once per outer row, though modern optimizers often decorrelate it into a join.
Subquery (FROM, Derived Table)	SELECT dept, avg_sal FROM (SELECT dept, AVG(salary) AS avg_sal FROM employees GROUP BY dept) t	• A subquery in the FROM clause • must have a table alias in most dialects • computed once and referenced like a table.
CTE (Common Table Expression)	WITH ranked AS (SELECT *, RANK() OVER (ORDER BY salary DESC) AS rk FROM employees) SELECT * FROM ranked WHERE rk <= 10	• Named temporary result set introduced with <code>WITH</code> and scoped to the statement it's attached to • can be referenced more than once by the query that follows it.
Recursive CTE	WITH RECURSIVE org AS (SELECT id, name, manager_id FROM employees WHERE manager_id IS NULL UNION ALL SELECT e.id, e.name, e.manager_id FROM employees e JOIN org o ON e.manager_id = o.id) SELECT * FROM org	• CTE that references itself to traverse hierarchical or graph data (org charts, bills of materials) • requires an anchor member, <code>UNION ALL</code> (or <code>UNION</code>) with a recursive member, and a condition that eventually returns no rows.

Technique	Example	Description
SEARCH / CYCLE Clause	<pre>WITH RECURSIVE search_graph(id, link, depth) AS (SELECT g.id, g.link, 1 FROM graph g UNION ALL SELECT g.id, g.link, sg.depth + 1 FROM graph g, search_graph sg WHERE g.id = sg.link) CYCLE id SET is_cycle USING path SELECT * FROM search_graph</pre>	• SEARCH DEPTH FIRST / BREADTH FIRST adds a sort column for a recursive CTE's traversal order • CYCLE auto-tracks visited rows so a graph with loops terminates instead of running forever.
Data-Modifyin g CTE	<pre>WITH moved_rows AS (DELETE FROM products WHERE "date" < '2010-11-01' RETURNING *) INSERT INTO products_log SELECT * FROM moved_rows</pre>	• Runs INSERT , UPDATE , DELETE or MERGE inside WITH , chaining several writes in one statement • only the RETURNING output can be read downstream, and each sub-statement runs exactly once regardless of whether that output is used.
MATERIALIZED D / NOT MATE RIALIZED	<pre>WITH w AS NOT MATERIALIZED (SELECT * FROM big_table) SELECT * FROM w AS w1 JOIN w AS w2 ON w1.key = w2.ref</pre>	• Optimizer hint on a CTE: MATERIALIZED forces it to compute once as a temporary result, NOT MATERIALIZED folds it into the parent query • PostgreSQL 12+ auto-folds a single-reference, side-effect-free CTE unless told otherwise.

TABLE 10

Set Operations

Set operators combine the rows of two or more **SELECT** queries that return the same number of columns with compatible data types. The result uses the column names from the first query, any **ORDER BY** must come at the very end of the combined statement, and **NULL** s are treated as equal for the purposes of duplicate elimination, a deliberate exception to SQL's usual three-valued logic.

Operation	Example	Description
UNION	<pre>SELECT city FROM customers UNION SELECT city FROM suppliers</pre>	• Combines result sets and removes duplicates (same logic as DISTINCT) • slower than UNION ALL because of the extra deduplication step.
UNION ALL	<pre>SELECT product_id FROM orders_2024 UNION ALL SELECT product_id FROM orders_2025</pre>	• Combines result sets including all duplicates • faster than UNION , so the canonical guidance is to default to UNION ALL and only use UNION when duplicates actually need removing.
INTERSECT	<pre>SELECT customer_id FROM orders_jan INTERSECT SELECT customer_id FROM orders_feb</pre>	• Returns rows that appear in both result sets • result is deduplicated; some dialects (e.g. PostgreSQL) also support INTERSECT ALL to keep duplicate matches.
EXCEPT (MINUS)	<pre>SELECT customer_id FROM customers EXCEPT SELECT customer_id FROM orders</pre>	• Returns rows in the first set that do not appear in the second • EXCEPT is the ANSI name (PostgreSQL, SQL Server, MySQL 8.0.31+); Oracle uses MINUS • result is deduplicated.
INTERSECT ALL / EXCEPT ALL	<pre>SELECT product_id FROM cart INTERSECT ALL SELECT product_id FROM warehouse_stock</pre>	• Keeps duplicate matches instead of deduplicating like plain INTERSECT / EXCEPT • both queries must still be union compatible (same column count, compatible types).

TABLE 11

Window Functions: Ranking

The ranking family assigns a position number to each row inside an ordered partition. The differences between them are entirely about how ties and gaps are handled, so picking the wrong one quietly changes results. Every ranking function requires `ORDER BY` inside its `OVER` clause; `PARTITION BY` is optional and restarts the numbering for each group.

Function	Example	Description
ROW_NUMBER()	<code>ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC)</code>	- Assigns a unique sequential integer per row within a partition - ties receive different numbers (broken by <code>ORDER BY</code> or arbitrarily).
RANK()	<code>RANK() OVER (ORDER BY score DESC)</code>	- Assigns rank with gaps after ties: two rows tied at rank 2 are both rank 2, the next row is rank 4 - competition-style ranking.
DENSE_RANK()	<code>DENSE_RANK() OVER (ORDER BY score DESC)</code>	- Like <code>RANK</code> but no gaps: two rows tied at rank 2 are both rank 2 - next row is rank 3.
NTILE()	<code>NTILE(4) OVER (ORDER BY salary)</code>	- Divides the ordered partition into N buckets as equal as possible - if rows don't divide evenly the earliest buckets get the extra rows.
PERCENT_RANK()	<code>PERCENT_RANK() OVER (ORDER BY salary)</code>	- Relative rank $(\text{rank} - 1) / (\text{total rows} - 1)$, range 0 to 1 inclusive - the first row is always 0.0.
CUME_DIST()	<code>CUME_DIST() OVER (ORDER BY salary)</code>	- Cumulative distribution: rows with value $\leq$ current row, divided by total rows - range 1/N to 1 - tied rows share the same value.

TABLE 12

Window Functions: Aggregate & Analytical

Window functions compute a value for each row using a *window* of related rows, without collapsing the result the way `GROUP BY` does. The window is shaped by three parts of the `OVER` clause: `PARTITION BY` (which rows belong together), `ORDER BY` (their sequence), and the optional frame (`ROWS` / `RANGE`, which of those rows are visible from the current one). Most window-function surprises come from the frame: adding `ORDER BY` without an explicit frame defaults to `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which silently turns whole-partition aggregates into running ones and makes `LAST_VALUE` / `NTH_VALUE` return the wrong row.

Function	Example	Description
SUM() OVER()	<code>SUM(amount) OVER (PARTITION BY dept ORDER BY date)</code>	Running total within a partition when <code>ORDER BY</code> is present - without <code>ORDER BY</code>, returns the partition total beside every row.
AVG() OVER()	<code>AVG(salary) OVER (PARTITION BY department_id)</code>	Partition mean on every row when no <code>ORDER BY</code> is given - adding <code>ORDER BY</code> turns it into a running average.
COUNT() OVER()	<code>COUNT(*) OVER (PARTITION BY customer_id)</code>	- Returns the number of rows in the partition beside every detail row - useful for showing a per-group total without <code>GROUP BY</code>.
LAG()	<code>LAG(close_price, 1, 0) OVER (ORDER BY date)</code>	- Returns the value offset rows before the current row - the first row returns <code>NULL</code> unless a default is given <code>IGNORE NULLS</code> skips nulls when locating the row; <code>RESPECT NULLS</code> is the default.
LEAD()	<code>LEAD(close_price, 1, 0) OVER (ORDER BY date)</code>	- Returns the value offset rows after the current row - the last row returns <code>NULL</code> unless a default is given <code>IGNORE NULLS</code> skips nulls when locating the row; <code>RESPECT NULLS</code> is the default.
FIRST_VALUE()	<code>FIRST_VALUE(salary) OVER (PARTITION BY dept ORDER BY hire_date)</code>	- Returns the value at the first row of the window frame - the default frame starts at the partition's first row <code>IGNORE NULLS</code> skips nulls when picking that value; <code>RESPECT NULLS</code> is the default.

Function	Example	Description
LAST_VALUE()	LAST_VALUE(salary) OVER (PARTITION BY dept ORDER BY hire_date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)	- Returns the value at the last row of the window frame - default frame stops at the current row; add ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING for the true last value IGNORE NULLS skips nulls; RESPECT NULLS is the default.
NTH_VALUE()	NTH_VALUE(salary, 2) OVER (PARTITION BY dept ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)	- Returns the value at the nth row of the window frame (counting from 1) - returns NULL if n exceeds the frame, so expand the frame to UNBOUNDED FOLLOWING when you need access to rows past the current position.
WINDOW clause	SELECT name, RANK() OVER w FROM employees WINDOW w AS (PARTITION BY dept ORDER BY salary DESC)	- Defines a named window specification that multiple window functions in the same query can reference via OVER w - removes duplication when several aggregates share one partition / order.

TABLE 13

Window Frame Specification

The frame clause narrows a window function's view to a subset of the partition relative to the current row, controlling exactly which rows feed running totals, moving averages, and lookups like **LAST_VALUE()**. The most important and most surprising rule: when **ORDER BY** is present but no frame is specified, the default is **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** (peer-based, not row-based), which silently fuses tied rows into a single running-sum step.

Clause	Example	Description
ROWS BETWEEN ... AND ...	SUM(amount) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)	- Frame defined by physical row offsets, counting rows regardless of their ORDER BY values - preferred for fixed-size moving calculations like 7-day windows
RANGE BETWEEN ... AND ...	SUM(amount) OVER (ORDER BY date RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)	- Frame defined by logical value ranges: all rows that are ORDER BY peers of the current row are included together - this is the default frame when ORDER BY is present and no frame clause is given
GROUPS BETWEEN ... AND ...	SUM(amount) OVER (ORDER BY date GROUPS BETWEEN 2 PRECEDING AND CURRENT ROW)	- Frame defined by peer groups: offsets count distinct ORDER BY values (ties count as one step), not individual rows - lets duplicate ordering keys advance the frame together instead of one row at a time
UNBOUNDED PRECEDING	ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	Frame starts at the first row of the current partition , used for cumulative aggregations from the partition start
UNBOUNDED FOLLOWING	ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING	Frame extends to the last row of the current partition , required for LAST_VALUE() and NTH_VALUE() to see all subsequent rows in standard SQL
N PRECEDING / N FOLLOWING	ROWS BETWEEN 3 PRECEDING AND 1 FOLLOWING	- Frame spans N rows before to M rows after the current row - in ROWS mode the offset must be a non-negative integer; in RANGE mode it requires exactly one numeric or date ORDER BY column
CURRENT ROW	ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	- In ROWS mode means just the current row; in RANGE/GROUPS mode it spans all peer rows with the same ORDER BY value - this peer behavior is what produces the running-total surprise on duplicate ordering keys
RANGE ... INTERVAL PRECEDING / FOLLOWING	SUM(amount) OVER (ORDER BY order_date RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT ROW)	- In RANGE mode, when the ORDER BY column is a date or timestamp, the offset is an interval instead of a plain number - builds true time-based windows, like a trailing 7-day total, even with gaps between rows

Clause	Example	Description
EXCLUDE CURRENT ROW / GROUP / TIES / NO OTHERS	<code>SUM(amount) OVER (ORDER BY amount RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW EXCLUDE TIES)</code>	- Removes rows around the current row from an otherwise-included frame CURRENT ROW drops just the current row, GROUP drops it and its peers, TIES drops its peers but keeps the current row, NO OTHERS is the default

TABLE 14

Advanced Grouping and Pivoting

GROUP BY extensions add multi-level subtotals and grand totals in a single pass over the data, replacing stacked `UNION ALL` queries with concise, optimizer-friendly syntax. ROLLUP gives strict hierarchical subtotals, CUBE gives every combination, and GROUPING SETS lets you cherry-pick exactly the sets you want; `GROUPING()` and `GROUPING_ID()` then let you tell super-aggregate rows apart from rows that happen to contain real NULLs. The same reporting territory covers reshaping rows into columns and back with PIVOT, UNPIVOT and their portable, CASE-based equivalents.

Extension	Example	Description
ROLLUP	<code>SELECT year, quarter, SUM(sales) FROM revenue GROUP BY ROLLUP(year, quarter)</code>	- Generates hierarchical subtotals: produces N+1 grouping sets for N columns (each prefix of the column list plus the grand total) - e.g., totals by (year, quarter), (year), and grand total in one query.
CUBE	<code>SELECT dept, year, SUM(revenue) FROM sales GROUP BY CUBE(dept, year)</code>	- Generates every subset of the listed columns: 2^N grouping sets for N columns (the full power set) - includes every cross-tabulation plus the grand total.
GROUPING SETS	<code>GROUP BY GROUPING SETS ((dept, year), (dept), (year), ())</code>	Explicit control over which grouping combinations to compute: no implicit hierarchy, no full crossing - avoids paying for CUBE combinations you don't need.
GROUPING()	<code>SELECT GROUPING(dept), dept, SUM(sal) FROM employees GROUP BY ROLLUP(dept)</code>	- Returns 1 for super-aggregate rows (where the column was rolled up to NULL) and 0 for normal rows - the only reliable way to distinguish ROLLUP/CUBE nulls from actual NULL values in the data.
GROUPING_ID()	<code>SELECT GROUPING_ID(dept, year), dept, year, SUM(sal) FROM employees GROUP BY ROLLUP(dept, year)</code>	- Returns an integer bitmap where each bit is the <code>GROUPING()</code> value for that column: identifies the row's grouping level in a single value - cleaner than multiple <code>GROUPING()</code> calls when you need to label, sort, or filter by level.
PIVOT	<code>SELECT dept, [2023], [2024], [2025] FROM sales_data PIVOT (SUM(amount) FOR year IN ([2023],[2024],[2025])) AS pvt</code>	Rotates row values into columns: aggregates a measure and spreads one column's distinct values into new columns - An aggregate function is always required; values outside the <code>IN</code> list are dropped.
UNPIVOT	<code>SELECT dept, year, amount FROM wide_table UNPIVOT (amount FOR year IN ([2023],[2024],[2025])) AS unpvt</code>	Reverses a pivot: turns column headers back into row values to normalize wide tables - In SQL Server, <code>NULL</code>-valued input cells produce no output row (Oracle exposes <code>INCLUDE NULLS</code> / <code>EXCLUDE NULLS</code>).
Conditional Aggregation	<code>SELECT dept, SUM(CASE WHEN year=2024 THEN sales END) AS [2024], SUM(CASE WHEN year=2025 THEN sales END) AS [2025] FROM sales GROUP BY dept</code>	Portable alternative to PIVOT: <code>CASE</code> inside aggregate functions, unchanged on SQL Server, PostgreSQL, MySQL, Oracle, SQLite - Column list is fixed at write time; wrap it in dynamic SQL to build the <code>CASE</code> list when values aren't known in advance.

Extension	Example	Description
crosstab()	<pre>SELECT * FROM crosstab('SELECT dept, year, amount FROM sales ORDER BY 1,2') AS ct(dept text, y2024 numeric, y2025 numeric)</pre>	• PostgreSQL's <code>tablefunc</code> contrib function for true row-to-column pivoting, since core PostgreSQL has no <code>PIVOT</code> operator • The source query must return one row-name, one category and one value column; output columns are named in the <code>FROM</code> clause.

TABLE 15

String Functions

String functions transform, inspect, and assemble text values directly inside SQL. Their behavior, especially around `NULL`, trailing spaces, and which dialect supports which syntax, varies meaningfully between SQL Server, PostgreSQL, MySQL, and Oracle, so the same expression can return different results depending on the engine.

Function	Example	Description
CONCAT()	<pre>CONCAT(first_name, ' ', last_name)</pre>	• Joins strings end-to-end • in SQL Server and MySQL, <code>NULL</code> arguments are converted to empty strings and ignored • PostgreSQL/Oracle/SQLite also support the <code> </code> operator, which by contrast returns <code>NULL</code> if any operand is <code>NULL</code>.
Concatenation Operator ()	<pre><code>first_name ' ' last_name</code></pre>	• Concatenates strings with an infix operator instead of a function call • returns <code>NULL</code> if either operand is <code>NULL</code>, unlike <code>CONCAT()</code> • PostgreSQL, Oracle, and SQLite; SQL Server 2025 adds it too.
CONCAT_WS()	<pre>CONCAT_WS(' ', city, state, country)</pre>	• Concatenates with a separator: the first argument • ignores <code>NULL</code> arguments and does not emit the separator for them, so the separator only ever appears between non-NULL values; SQL Server 2017+, PostgreSQL, MySQL.
UPPER() / LOWER()	<pre>UPPER(last_name)</pre>	• Converts a string to uppercase or lowercase according to the database's locale • case folding is locale-sensitive: the Turkish dotted/dotless <code>I</code> is a classic gotcha.
TRIM() / LTRIM() / RTRIM()	<pre>TRIM(LEADING ' ' FROM name)</pre>	• Removes leading, trailing, or both spaces (or specified characters) • standard SQL <code>TRIM</code> supports <code>LEADING</code> / <code>TRAILING</code> / <code>BOTH</code> • SQL Server gained <code>TRIM</code> in 2017; the <code>LEADING</code> / <code>TRAILING</code> / <code>BOTH</code> keywords require SQL Server 2022 with compatibility level 160.
SUBSTRING() / SUBSTR()	<pre>SUBSTRING(email, 1, CHARINDEX('@', email) - 1)</pre>	• Extracts a substring given a 1-based start position and optional length • Oracle's <code>SUBSTR</code> also accepts a negative start to count from the string's end; SQL Server's <code>SUBSTRING</code> does not.
LEFT() / RIGHT()	<pre>LEFT(phone, 3) RIGHT(zip_code, 4)</pre>	• Extracts N characters from the left or right end of a string • a convenient shorthand for <code>SUBSTRING</code>.
LEN() / LENGTH()	<pre>WHERE LEN(description) > 500</pre>	• Returns the number of characters in a string • SQL Server's <code>LEN</code> silently excludes trailing spaces, use <code>DATALLENGTH</code> for the byte length including them • PostgreSQL/MySQL's <code>LENGTH</code> counts everything.
REPLACE()	<pre>REPLACE(phone, '-', '')</pre>	• Substitutes all occurrences (not just the first) of a substring with another string • in SQL Server the match honors the column's collation, which is case-insensitive by default.
REVERSE()	<pre>REVERSE(phone)</pre>	Reverses the order of the characters in a string.
CHARINDEX() / INSTR() / POSITION()	<pre>CHARINDEX('@', email)</pre>	• Returns the 1-based character position of a substring, or <code>0</code> if not found (never <code>NULL</code> unless an argument is <code>NULL</code>) • <code>CHARINDEX</code> in SQL Server, <code>INSTR</code> in Oracle/MySQL, <code>POSITION(substr IN str)</code> in PostgreSQL.

Function	Example	Description
STRING_SPLIT()	<code>SELECT value FROM STRING_SPLIT('a,b,c', ',')</code>	• Splits a string on a single-character delimiter and returns a one-column table • SQL Server 2016+ • output order is not guaranteed unless you pass <code>enable_ordinal => 1</code> (SQL Server 2022+) and <code>ORDER BY ordinal</code> • PostgreSQL uses <code>string_to_table</code> / <code>regexp_split_to_table</code> .
SPLIT_PART()	<code>SPLIT_PART(email, '@', 2)</code>	• Returns one field from a delimiter-split string, chosen by position (1-based; negative counts from the end) • PostgreSQL, and Redshift/Snowflake; SQL Server has no direct equivalent (use <code>STRING_SPLIT</code> with <code>ordinal</code>).
REPEAT() / REPLICATE()	<code>REPEAT('=', 20)</code> <code>REPLICATE('=', 20)</code>	• Repeats a string a specified number of times • <code>REPEAT()</code> in PostgreSQL/MySQL, <code>REPLICATE()</code> in SQL Server.
LPAD() / RPAD()	<code>LPAD(CAST(id AS VARCHAR), 8, '0')</code> <code>RPAD(name, 20, ' ')</code>	• Pads a string to a length by prepending/appending fill characters, truncating if already longer • PostgreSQL/MySQL/Oracle only; SQL Server's workaround is <code>RIGHT('00000000' + CAST(id AS VARCHAR), 8)</code> .
TRANSLATE()	<code>TRANSLATE('12345', '143', 'ax')</code>	• Replaces each character matching one in a "from" set with the corresponding character in a "to" set, never re-scanning a character already replaced • PostgreSQL allows <code>from</code> longer than <code>to</code> ; SQL Server requires equal lengths or raises an error.
INITCAP()	<code>INITCAP('john SMITH')</code>	• Capitalizes the first letter of each word and lowercases the rest, where a word is a run of alphanumeric characters • PostgreSQL and Oracle only, no direct equivalent in SQL Server.
FORMAT()	<code>FORMAT(amount, 'C2', 'en-US')</code>	• Formats a value as a string using .NET format strings with optional culture (SQL Server only) • handles currency and number formatting with locale awareness, but relies on the CLR and is slower than <code>CONVERT</code> on large row counts.

TABLE 16

Pattern Matching

SQL provides several pattern-matching predicates for filtering rows by text shape, from the simple wildcard-based `LIKE` to dialect-specific regex operators and full-text search. Which one you reach for depends on how complex the pattern is and which engine you're on.

Technique	Example	Description
LIKE	<code>WHERE name LIKE 'A%'</code> <code>WHERE code LIKE '_-[0-9][0-9]'</code>	• <code>%</code> matches any sequence of characters; <code>_</code> matches exactly one character • SQL Server also supports character classes <code>[a-z]</code> .
ILIKE	<code>WHERE name ILIKE '%smith%'</code>	• Case-insensitive LIKE • PostgreSQL only • equivalent to <code>LIKE</code> with <code>LOWER()</code> .
NOT LIKE	<code>WHERE email NOT LIKE '%example.com'</code>	Inverts pattern match: returns rows where the value does not match the pattern.
REGEXP_LIKE / SIMILAR TO / ~ (tilde)	<code>REGEXP_LIKE(phone, '^d{3}-d{4}\$')</code> (Oracle/SQL Server 2025) <code>phone ~ '^d{3}-d{4}\$'</code> (PostgreSQL)	• Full regular expression matching, returns true/false • <code>REGEXP_LIKE</code> in Oracle and now SQL Server 2025 (2025); <code>~</code> in PostgreSQL; <code>SIMILAR TO</code> is the SQL standard form.
REGEXP_REPLACE	<code>REGEXP_REPLACE(product_name, '[ae]', 'X', 1, 0, 'i')</code>	• Substitutes text matching a regex pattern with a replacement • also native in Oracle and PostgreSQL 15+.
REGEXP_SUBSTR / REGEXP_INSTR	<code>REGEXP_SUBSTR(email, '@(.+)\$', 1, 1, 'i', 1)</code> <code>REGEXP_INSTR(description, '\d+')</code>	• <code>REGEXP_SUBSTR</code> extracts the matching substring; <code>REGEXP_INSTR</code> returns its position • both native in Oracle, PostgreSQL 15+ and SQL Server 2025.

Technique	Example	Description
REGEXP_COUNT	<code>REGEXP_COUNT(product_name, 'a')</code>	- Counts how many times a regex pattern matches inside a string - also native in Oracle and PostgreSQL 15+.
CONTAINS	<code>WHERE CONTAINS(description, 'fast AND reliable')</code>	Full-text search predicate (SQL Server) - requires a full-text index; supports boolean operators and proximity searches.
ESCAPE clause	<code>WHERE path LIKE '100\%' ESCAPE '\'</code>	Treats special LIKE metacharacters (<code>%</code> , <code>_</code>) as literals when prefixed with the escape character.

TABLE 17

Date & Time Functions

Date and time handling varies more between dialects than almost any other area of SQL, function names, return types, and time-zone semantics all differ. The functions below cover the common needs: getting the current moment, doing arithmetic on dates, extracting parts, converting time zones, truncating to a unit, and formatting for display. Watch for two traps that bite even experienced developers: `DATEDIFF` counts boundary crossings rather than elapsed time, and `NOW()` in PostgreSQL returns the *transaction* start time, not the actual clock.

Function	Example	Description
GETDATE() / NOW() / CURRENT_TIMESTAMP	<code>SELECT GETDATE()</code>	- Returns the current date and time <code>GETDATE()</code> in SQL Server returns server local time; use <code>GETUTCDATE()</code> or <code>SYSUTCDATETIME()</code> for UTC <code>NOW()</code> in PostgreSQL/MySQL and <code>CURRENT_TIMESTAMP</code> (ANSI) work across dialects.
CURRENT_DATE	<code>SELECT CURRENT_DATE</code>	- Returns the current date only, with no time component - ANSI standard, long available in PostgreSQL, MySQL and Oracle; added to SQL Server 2025.
DATEADD() / DATE_ADD() / INTERVAL	<code>DATEADD(month, 3, hire_date)</code> <code>hire_date + INTERVAL '3 months'</code>	Adds a time interval to a date; pass a negative number to subtract <code>DATEADD</code> in SQL Server, <code>DATE_ADD</code> in MySQL, <code>+ INTERVAL</code> in PostgreSQL/Oracle.
DATEDIFF()	<code>DATEDIFF(day, start_date, end_date)</code>	- Returns the number of datepart boundaries crossed between two dates, not strictly elapsed time <code>DATEDIFF(year, '2024-12-31', '2025-01-01')</code> returns <code>1</code> even though the dates are 1 day apart - SQL Server / MySQL syntax; PostgreSQL uses <code>EXTRACT</code> or subtraction.
DATEPART() / EXTRACT()	<code>DATEPART(year, hire_date)</code> <code>EXTRACT(YEAR FROM hire_date)</code>	- Returns a specific date component (year, month, day, hour, etc.) as an integer <code>DATEPART</code> is T-SQL; <code>EXTRACT</code> is the ANSI-standard form supported by PostgreSQL, MySQL, Oracle, and SQL Server 2022+.
AT TIME ZONE	<code>timestamp '2001-02-16 20:38:40' AT TIME ZONE 'America/Denver'</code>	Converts a timestamp between time zones, attaching or shifting its UTC offset - PostgreSQL takes IANA zone names (<code>'America/Denver'</code>); SQL Server's <code>AT TIME ZONE</code> uses Windows time zone names instead.
DATE_TRUNC()	<code>DATE_TRUNC('month', created_at)</code>	Truncates a timestamp to a specified precision (year, quarter, month, day, hour), ideal for grouping by month or quarter - PostgreSQL / Snowflake / Redshift; SQL Server 2022+ provides <code>DATETRUNC()</code> .
CAST() (to date/time)	<code>CAST('2025-01-15' AS DATE)</code>	Converts a string to a date/datetime value (ANSI standard, works across dialects) - In SQL Server, ambiguous formats like <code>'01/02/2025'</code> depend on session <code>SET DATEFORMAT</code> / language; ISO 8601 (<code>YYYY-MM-DD</code>) is always safe.
DATEFROMPARTS() / MAKE_DATE()	<code>DATEFROMPARTS(2025, 1, 15)</code> <code>MAKE_DATE(2025, 1, 15)</code>	Builds a date value from separate year, month and day integers, avoiding string-parsing ambiguity <code>DATEFROMPARTS</code> in SQL Server 2012+; <code>MAKE_DATE</code> in PostgreSQL.

Function	Example	Description
TO_CHAR() / FORMAT()	TO_CHAR(order_date, 'YYYY-MM') FORMAT(order_date, 'yyyy-MM')	• Formats a date as a string with a template pattern, useful for display or grouping by year-month • TO_CHAR in PostgreSQL/Oracle/Snowflake; FORMAT in SQL Server 2012+ and MySQL.
EOMONTH() / LAST_DAY()	EOMONTH(GETDATE()) LAST_DAY(order_date)	• Returns the last day of the month for a given date (as a date, with the time component dropped) • EOMONTH's optional second argument adds/subtracts months before computing month-end • EOMONTH in SQL Server 2012+; LAST_DAY in MySQL/MariaDB and Oracle.
DATE_BUCKET()	DATE_BUCKET(month, 1, order_date)	• Groups timestamps into fixed-width buckets (every N days, hours, minutes...) measured from an origin point, for time-series aggregation • SQL Server 2022+.

TABLE 18

Numeric Functions

Numeric functions handle the arithmetic, rounding, and randomness work that lives outside of plain $+$ $-$ $*$ $/$. Dialect quirks bite hard here: what **LOG(100)** returns, whether **ROUND** rounds halves away from zero or to-even, and whether **RAND()** re-runs per row all change between vendors.

Function	Example	Description
ROUND()	ROUND(123.45, -1) → 120	• Rounds to N decimal places • negative N rounds to tens, hundreds, etc. • SQL Server and PostgreSQL numeric break ties away from zero.
FLOOR() / CEILING()	FLOOR(-4.7) → -5 CEILING(4.2) → 5	• FLOOR returns the largest integer $\leq$ value (rounds toward $-\infty$) • CEILING returns the smallest integer $\geq$ value (rounds toward $+\infty$).
ABS()	ABS(balance - target)	• Returns the absolute value, stripping the sign • Can overflow on the minimum signed integer (e.g. ABS(-2147483648) in SQL Server).
POWER() / SQRT()	POWER(2, 10) → 1024 SQRT(144) → 12	• POWER(x, y) raises x to the y-th power • SQRT returns the square root; negative input raises a domain error in most engines.
MOD() / %	SELECT 17 % 5 → 2 MOD(17, 5) → 2	• Returns the remainder after division • % in SQL Server/PostgreSQL, MOD() in Oracle/MySQL/PostgreSQL • In Postgres the result inherits the sign of the dividend.
Integer Division (/)	5 / 2 → 2 (-5) / 2 → -2 5.0 / 2 → 2.5	• Integer divided by integer truncates toward zero, it does not round • Cast one side to a decimal or float type (5.0 / 2) to get a fractional result.
LOG() / LOG10() / LN()	LOG(EXP(1)) → 1 (SQL Server) LOG(100) → 2 (PostgreSQL)	• SQL Server / MySQL: LOG(x) = natural log; LOG(x, base) for arbitrary base in SQL Server • PostgreSQL: LOG(x) = base-10; use LN(x) for natural log • LOG10 is always base-10.
SIGN()	SIGN(balance) → -1, 0, or 1	• Returns -1, 0, or +1 for negative, zero, or positive input • useful for directional logic without CASE.
RAND() / RANDOM()	SELECT RAND() (SQL Server) SELECT RANDOM() (PostgreSQL)	• Returns a pseudo-random float between 0 and 1 • RAND in SQL Server/MySQL, RANDOM() in PostgreSQL • In SQL Server, RAND() evaluates once per query, use NEWID() for per-row randomness.
TRUNC() / TRUNCATE()	TRUNC(3.987, 2) → 3.98 TRUNC(-4.7) → -4	• Chops (does not round) to N decimal places • Oracle/PostgreSQL use TRUNC, MySQL uses TRUNCATE • Differs from FLOOR for negatives (TRUNC rounds toward zero, FLOOR toward $-\infty$).

TABLE 19

Data Types

The type a column is declared with decides its storage size, its range and whether values are stored exactly. Examples below use PostgreSQL syntax; SQL Server and MySQL name several of these types differently, noted where they do.

Concept	Example	Description
Integer Types	<code>id INTEGER, hits BIGINT, age SMALLINT</code>	• SMALLINT (2 bytes), INTEGER (4 bytes), BIGINT (8 bytes) hold whole numbers of increasing range • same names in SQL Server; MySQL adds TINYINT and MEDIUMINT
Character Types	<code>name VARCHAR(50), bio TEXT</code>	• VARCHAR(n) is variable-length up to n characters, CHAR(n) is fixed-length and blank-padded, TEXT is unlimited length • SQL Server's Unicode-storing equivalents are NVARCHAR and NCHAR
Exact Numeric: NUMERIC / DECIMAL	<code>price NUMERIC(10,2)</code>	• User-specified precision and scale, stored and rounded exactly • NUMERIC and DECIMAL are equivalent
Date/Time Types	<code>created_at TIMESTAMP WITH TIME ZONE DEFAULT now()</code>	• DATE, TIME, TIMESTAMP and TIMESTAMP WITH TIME ZONE store calendar values at microsecond resolution • SQL Server's higher-precision equivalent is DATETIME2
Boolean	<code>is_active BOOLEAN DEFAULT TRUE</code>	• Stores true, false, or unknown (NULL) in 1 byte • SQL Server has no boolean type and uses BIT (0/1) instead
UUID	<code>id UUID DEFAULT uuidv7()</code>	• 128-bit globally unique identifier • PostgreSQL 18 adds <code>uuidv7()</code> for timestamp-ordered UUIDs alongside <code>uuidv4()</code> • SQL Server's equivalent is UNIQUEIDENTIFIER, a 16-byte GUID
JSON / JSONB	<code>SELECT '{"active": false}':::jsonb;</code>	• json stores an exact text copy, reparsed on each use; jsonb stores a decomposed binary form that supports indexing and is normally preferred • SQL Server 2025 adds a native binary JSON type
Approximate Numeric: REAL / DOUBLE PRECISION	<code>SELECT round(2.5::double precision);</code>	• Variable-precision, inexact floating-point types (6 vs 15 decimal digits) • prefer NUMERIC when exact rounding matters
Interval	<code>SELECT now() + INTERVAL '7 days';</code>	Stores a time span (years down to microseconds), optionally restricted to a field range such as <code>YEAR TO MONTH</code>
Binary Types: BYTEA / VARBINARY	<code>SELECT 'abc':::bytea;</code>	• Variable-length raw byte storage, distinct from character strings • SQL Server's equivalent is <code>VARBINARY(n max)</code>, with fixed-length BINARY(n)

TABLE 20

Data Type Conversion

Explicit conversion functions turn one data type into another and decide what happens when the input doesn't fit. The big split is **error vs NULL on failure**: `CAST` / `CONVERT` / `PARSE` raise errors, while `TRY_CAST` / `TRY_CONVERT` / `TRY_PARSE` return NULL, much safer for dirty user input. Choosing the right one (and pairing column types in `WHERE` filters) also has real performance impact, because implicit conversions can stop SQL Server from using an index.

Method	Example	Description
CAST()	<code>CAST('3.14' AS DECIMAL(10,2))</code>	• ANSI-standard explicit type conversion, works in T-SQL, PostgreSQL, MySQL, Oracle, Snowflake, DuckDB • raises an error if the conversion fails.
:: (cast operator)	<code>'3.14':::DECIMAL(10,2)</code>	• PostgreSQL's historical cast syntax, equivalent to <code>CAST(expression AS type)</code> • not portable to other engines.
TRY_CAST()	<code>TRY_CAST(user_input AS INT)</code>	• Safe CAST (SQL Server 2012+), returns NULL instead of an error when the conversion fails • ideal for validating user input or dirty data.

Method	Example	Description
CONVERT()	<code>CONVERT(VARCHAR, GETDATE(), 120)</code>	• SQL Server type conversion with an optional <code>style</code> argument for date/number formatting (e.g., 120 = ODBC canonical <code>yyyy-mm-dd hh:mi:ss</code>) • MySQL <code>CONVERT</code> has no style code.
TRY_CONVERT()	<code>TRY_CONVERT(DATE, date_string, 101)</code>	• Safe CONVERT (SQL Server 2012+), returns NULL on conversion failure • supports the same style codes as <code>CONVERT</code> .
Implicit Conversion	<code>WHERE varchar_col = 123</code>	• Automatic conversion SQL Server applies when comparing or combining mismatched types, following data type precedence • can force an index scan instead of an index seek on the converted column.
TO_NUMBER() / TO_DATE() / TO_TIMESTAMP()	<code>TO_DATE('05 Dec 2000', 'DD Mon YYYY')</code> <code>TO_NUMBER('12,454.8', '99G999D9')</code>	• PostgreSQL and Oracle format-model conversions • the explicit format string controls how the input is parsed.
PARSE()	<code>PARSE('15 Janvier 2025' AS DATE USING 'fr-FR')</code>	• SQL Server culture-aware string-to-date/number parse; raises an error on failure (use <code>TRY_PARSE</code> for the safe variant) • depends on the .NET CLR.
TRY_PARSE()	<code>TRY_PARSE('€1.234,56' AS MONEY USING 'de-DE')</code>	• Culture-aware string parsing to date/time or number types, returns NULL on failure • relies on the .NET CLR, so slower than <code>TRY_CAST</code> .
STR()	<code>STR(123.45, 6, 1)</code> returns <code>'123.5'</code>	• SQL Server float-to-character conversion with fixed length and decimal places • right-justifies with leading spaces and returns asterisks (<code>**</code>) when the length is too small.

TABLE 21

DDL Statements

Data Definition Language (DDL) shapes the database itself: creating, altering and dropping the tables, columns, schemas and other objects that hold your data. Several statements behave differently across engines: `TRUNCATE` is transactional in PostgreSQL and SQL Server but auto-commits in MySQL and Oracle, and rename syntax splits into ANSI `ALTER TABLE ... RENAME TO` vs SQL Server's `sp_rename` .

Statement	Example	Description
CREATE TABLE	<code>CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(255) UNIQUE)</code>	Creates a new table with column definitions, data types, and constraints.
CREATE TABLE AS SELECT	<code>CREATE TABLE active_users AS SELECT id, name FROM users WHERE active = true</code>	• Creates a table and fills it with the result of a query, computed once • the new table does not track later changes to the source table.
IF NOT EXISTS / IF EXISTS	<code>CREATE TABLE IF NOT EXISTS logs (id INT); DROP TABLE IF EXISTS logs;</code>	Suppresses the error when the object already exists (or is already absent), issuing a notice instead.
Temporary Table	<code>CREATE TEMP TABLE staging (id INT, payload TEXT)</code>	• Visible only in the current session • dropped automatically at session end, or optionally at end of transaction with <code>ON COMMIT</code> .
Identity Column	<code>CREATE TABLE people (id bigint GENERATED ALWAYS AS IDENTITY, name text)</code>	• Auto-generates key values from a hidden sequence (<code>ALWAYS</code> blocks manual overrides, <code>BY DEFAULT</code> allows them) • MySQL uses <code>AUTO_INCREMENT</code> , SQL Server uses <code>IDENTITY(seed, increment)</code> .

Statement	Example	Description
Generated Column	<code>CREATE TABLE people (height_cm numeric, height_in numeric GENERATED ALWAYS AS (height_cm / 2.54))</code>	- Computed from other columns in the same row, cannot be written to directly virtual (computed on read) by default since PostgreSQL 18; STORED computes and saves it at write time.
ALTER TABLE ADD COLUMN	<code>ALTER TABLE employees ADD COLUMN phone VARCHAR(20)</code>	Adds a new column to an existing table - existing rows get NULL unless a DEFAULT is specified.
ALTER TABLE MODIFY / ALTER COLUMN	<code>ALTER TABLE products ALTER COLUMN price DECIMAL(12,2)</code>	Changes a column's definition (data type, constraints) - syntax varies (MODIFY in MySQL, ALTER COLUMN in SQL Server / PostgreSQL).
ALTER TABLE RENAME COLUMN	<code>ALTER TABLE distributors RENAME COLUMN address TO city</code>	Renames a column without changing its data, type, or constraints.
ALTER TABLE RENAME	<code>ALTER TABLE customer RENAME TO customers</code>	Renames an existing table - syntax varies (RENAME TO in PostgreSQL / Oracle / MySQL, sp_rename in SQL Server).
ALTER TABLE DROP COLUMN	<code>ALTER TABLE users DROP COLUMN legacy_flag</code>	Permanently removes a column and its data - in PostgreSQL it's metadata-only, so disk space isn't reclaimed until the table is rewritten
DROP TABLE	<code>DROP TABLE IF EXISTS temp_staging</code>	Permanently deletes a table, all its data, indexes, and triggers IF EXISTS prevents an error if already absent; CASCADE removes dependent foreign-key constraints and views.
TRUNCATE TABLE	<code>TRUNCATE TABLE audit_log</code>	Removes all rows while preserving table structure - much faster than DELETE ; does not fire row-level DELETE triggers; rollback works in PostgreSQL / SQL Server but auto-commits in MySQL / Oracle.
CREATE SEQUENCE	<code>CREATE SEQUENCE order_id_seq START WITH 1 INCREMENT BY 1</code>	Defines a standalone number generator that tables can share, read with <code>nextval()</code> .
CREATE SCHEMA	<code>CREATE SCHEMA IF NOT EXISTS sales AUTHORIZATION app_user</code>	- Creates a namespace inside a database that groups tables, types, and functions - identically named objects can exist in different schemas
CREATE DATABASE	<code>CREATE DATABASE analytics</code>	Creates a new database on the server, a top-level container with its own schemas, users, and storage.

TABLE 22

Constraints & Keys

Constraints enforce data integrity at the column or table level, rejecting writes that would violate the rules you declare in `CREATE TABLE` or `ALTER TABLE` . Watch the NULL handling closely: UNIQUE, CHECK, and FOREIGN KEY each treat NULL differently, and the defaults differ across SQL Server, PostgreSQL, MySQL, and Oracle.

Constraint	Example	Description
PRIMARY KEY	<code>id INT PRIMARY KEY</code>	Uniquely identifies each row: enforces NOT NULL + UNIQUE together - one per table; may span multiple columns (composite PK) - creates a unique clustered index by default in SQL Server (override with PRIMARY KEY NONCLUSTERED); PostgreSQL has no clustered-index concept.
FOREIGN KEY	<code>FOREIGN KEY (dept_id) REFERENCES departments(id)</code>	Enforces referential integrity : the value must match a row in the referenced table or be NULL (FK columns are nullable unless you also add NOT NULL).
NOT NULL	<code>name VARCHAR(100) NOT NULL</code>	Rejects NULL values on INSERT and UPDATE - independent from DEFAULT : supplying an explicit NULL still raises an error even if a default exists.

Constraint	Example	Description
UNIQUE	<code>email VARCHAR(255) UNIQUE</code>	- Ensures all non-NULL values are distinct and creates a backing unique index - PostgreSQL / MySQL / Oracle allow multiple NULLs (NULL ≠ NULL); SQL Server's UNIQUE allows only one NULL.
CHECK	<code>salary DECIMAL CHECK (salary >= 0)</code>	Validates a Boolean expression on INSERT and UPDATE - NULL is treated as UNKNOWN, not FALSE: a CHECK constraint passes when the expression is TRUE or NULL, so NULLs slip through unless you also add NOT NULL.
DEFAULT	<code>created_at DATETIME DEFAULT GETDATE()</code>	Supplies a value when the column is omitted from INSERT or you write the DEFAULT keyword - an explicit NULL is NOT replaced by the default: it goes in as NULL (or errors if the column is NOT NULL).
ON DELETE CASCADE	<code>FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE</code>	Automatically deletes child rows when the referenced parent row is deleted - alternatives: SET NULL nullifies the child FK column, SET DEFAULT uses the child's default, NO ACTION / RESTRICT block the parent delete.
ON UPDATE CASCADE	<code>FOREIGN KEY (dept_id) REFERENCES departments(id) ON UPDATE CASCADE</code>	Propagates a change to the parent key into every matching child row - alternatives: SET NULL, SET DEFAULT; the default NO ACTION blocks the update if the change would break existing child references.
Named Constraint (CONSTRAINT name)	<code>price numeric CONSTRAINT positive_price CHECK (price > 0)</code>	Assigns an explicit name instead of a system-generated one, so error messages are clearer and the constraint can be altered or dropped by name later.
ALTER TABLE ADD / DROP CONSTRAINT	<code>ALTER TABLE distributors ADD CONSTRAINT distfk FOREIGN KEY (address) REFERENCES addresses (address); ALTER TABLE distributors DROP CONSTRAINT distfk;</code>	- Adds or removes a constraint on an existing table without recreating it ADD ... NOT VALID skips the initial table scan; a later VALIDATE CONSTRAINT checks existing rows without locking out concurrent writes.
DEFERRABLE INITIALLY DEFERRED	<code>UNIQUE (id) DEFERRABLE INITIALLY DEFERRED</code>	- Postpones the constraint check to the end of the transaction instead of after each statement - only UNIQUE, PRIMARY KEY, EXCLUDE and FOREIGN KEY constraints support it; NOT NULL and CHECK cannot be deferred.
UNIQUE NULLS NOT DISTINCT	<code>product_no integer UNIQUE NULLS NOT DISTINCT</code>	Makes a unique constraint treat NULL like any other value, so at most one NULL is allowed instead of PostgreSQL's default of treating every NULL as distinct.
WITHOUT OVERLAPS (temporal key)	<code>PRIMARY KEY (id, valid_at) WITHOUT OVERLAPS)</code>	- Checks the last key column for overlap instead of equality, enabling temporal primary keys and unique constraints (PostgreSQL 18) - the matching foreign key side uses PERIOD column_name instead.
NOT ENFORCED	<code>CHECK (price > 0) NOT ENFORCED</code>	- Declares a constraint the database will not check, leaving enforcement to the application (PostgreSQL 18) - useful as documentation when live checking is too costly; only supported for foreign key and CHECK constraints.

TABLE 23

DML Statements

Data Manipulation Language (DML) statements write to data rather than to the schema: **INSERT** adds rows, **UPDATE** modifies them, **DELETE** removes them, and **MERGE** does any combination of the three in one set-based statement. Every DML statement that modifies rows can be rolled back inside an explicit transaction, and every one of them silently affects every row in the table if you forget the **WHERE** clause.

Statement	Example	Description
INSERT INTO ... VALUES	<code>INSERT INTO orders (customer_id, amount, status) VALUES (42, 299.99, 'pending')</code>	Inserts one or more rows into a table using literal values - column list is optional, but if omitted the values are matched positionally against the table's declared column order.

Statement	Example	Description
Multi-row INSERT	<pre>INSERT INTO log (event, ts) VALUES ('start', NOW()), ('stop', NOW())</pre>	Inserts several rows in one statement with comma-separated VALUES lists, faster than issuing one INSERT per row.
INSERT INTO ... SELECT	<pre>INSERT INTO archive SELECT * FROM orders WHERE order_date < '2025-01-01'</pre>	• Bulk-inserts rows from a query result; columns are paired positionally between SELECT and the target • standard pattern for copying, archiving, or staging data.
INSERT ... ON CONFLICT (UPSERT)	<pre>INSERT INTO users (id, name) VALUES (1, 'Ada') ON CONFLICT (id) DO UPDATE SET name = EXCLUDED.name</pre>	• Inserts a row, or updates it instead when it collides with a unique or primary key • PostgreSQL/SQLite use ON CONFLICT ... DO UPDATE , MySQL/MariaDB use ON DUPLICATE KEY UPDATE .
UPDATE ... SET	<pre>UPDATE employees SET salary = salary * 1.1 WHERE department = 'Engineering'</pre>	• Modifies existing rows that match the WHERE condition • without WHERE every row is updated: a classic production-disaster mistake.
UPDATE with JOIN	<pre>UPDATE e SET e.dept_name = d.name FROM employees e JOIN departments d ON e.dept_id = d.id</pre>	• Updates rows using data from a joined table • syntax differs across dialects: T-SQL uses UPDATE ... FROM ... JOIN , MySQL uses UPDATE t1 JOIN t2 ... SET , PostgreSQL uses UPDATE ... FROM with the join condition in WHERE.
DELETE FROM	<pre>DELETE FROM sessions WHERE last_active < NOW() - INTERVAL '30 days'</pre>	• Deletes matching rows and logs each row removal individually • fires row-level triggers and can be rolled back; without WHERE it empties the table (but leaves the schema intact).
DELETE ... USING (DELETE with JOIN)	<pre>DELETE FROM orders o USING customers c WHERE o.customer_id = c.id AND c.status = 'banned'</pre>	• Deletes rows using data from another table in one statement instead of a subquery • PostgreSQL uses USING , T-SQL uses DELETE ... FROM ... JOIN , MySQL/MariaDB support a multi-table DELETE t1 FROM t1 JOIN t2 .
MERGE (UPSERT)	<pre>MERGE target AS t USING source AS s ON t.id = s.id WHEN MATCHED THEN UPDATE SET t.val = s.val WHEN NOT MATCHED THEN INSERT (id, val) VALUES (s.id, s.val) WHEN NOT MATCHED BY SOURCE THEN DELETE;</pre>	• Combines INSERT, UPDATE, and DELETE in one statement based on a match condition (added to PostgreSQL in v15) • requires a trailing semicolon in SQL Server; under concurrency, WITH (HOLDLOCK) is needed to prevent the well-known UPSERT race condition.
RETURNING / OUTPUT clause	<pre>INSERT INTO users (name) VALUES ('Ada') RETURNING id</pre>	• Returns column values from the affected rows without a separate SELECT • PostgreSQL/Oracle use RETURNING (with OLD / NEW aliases since PostgreSQL 18), SQL Server uses OUTPUT with inserted / deleted pseudo-tables.

TABLE 24

Indexes

Indexes are the single biggest performance lever in SQL: they turn full table scans into fast lookups by keeping a sorted, pointer-based structure (usually a B-tree) over chosen columns. They are pure redundancy though: every INSERT, UPDATE or DELETE that touches an indexed column also rewrites the index, so over-indexing is just as damaging as missing indexes, and column order in multi-column indexes is rarely a free choice. Whether an index actually gets used is confirmed with EXPLAIN, not assumed.

Concept	Example	Description
CREATE INDEX	<pre>CREATE INDEX idx_last_name ON customers (last_name)</pre>	Creates a non-unique index (B-tree by default) to speed up queries that filter, sort or join on the column(s).
EXPLAIN / EXPLAIN ANALYZE	<pre>EXPLAIN ANALYZE SELECT * FROM tenk1 WHERE unique1 = 42</pre>	• Shows the query plan the optimizer chose, including whether it used an index scan or a sequential scan • ANALYZE actually runs the query and adds real timings and row counts next to the estimates.

Concept	Example	Description
Sargable Predicate	<pre>WHERE order_date >= '2024-01-01' (sargable) WHERE YEAR(order_date) = 2024 (not sargable)</pre>	- A predicate is sargable when the optimizer can use an index to evaluate it directly - wrapping an indexed column in a function usually forces a full scan instead
CREATE UNIQUE INDEX	<pre>CREATE UNIQUE INDEX idx_email ON users (email)</pre>	- Enforces uniqueness on the indexed column(s) while accelerating lookups - functionally equivalent to a UNIQUE constraint, which is implemented as a unique index behind the scenes.
Composite Index	<pre>CREATE INDEX idx_name_dept ON employees (last_name, department_id)</pre>	- Index on multiple columns in a defined order, usable only when the query filters on a leftmost prefix of those columns - an index on (A, B) helps WHERE A=... and WHERE A=... AND B=... , but not WHERE B=... alone.
Covering Index (INCLUDE)	<pre>CREATE INDEX idx_cust ON orders (customer_id) INCLUDE (amount, order_date)</pre>	- Adds non-key columns to the index leaf via INCLUDE (SQL Server, PostgreSQL 11+) - query is satisfied entirely from the index, with no key lookup back to the table: a big read-path win for hot queries.
Clustered Index	<pre>CREATE CLUSTERED INDEX idx_order_id ON orders (order_id)</pre>	Physically orders the table rows by the index key; the table data is the index's leaf level - only one per table; SQL Server's PRIMARY KEY creates one by default, PostgreSQL has no equivalent (only a one-time CLUSTER command).
Non-clustered Index	<pre>CREATE NONCLUSTERED INDEX idx_dept ON employees (department_id)</pre>	- Maintains a separate structure with key values plus row locators back to the heap or clustered index - multiple allowed per table; fast for selective lookups but each non-covered query needs a key lookup.
Filtered Index (Partial Index)	<pre>CREATE INDEX idx_active ON users (email) WHERE active = 1</pre>	- Indexes only the rows matching a WHERE predicate (called a "filtered index" in SQL Server, "partial index" in PostgreSQL) - smaller and cheaper to maintain, useful when queries usually target a known subset.
Expression Index (Function-Based Index)	<pre>CREATE INDEX idx_lower_email ON users (lower(email))</pre>	- Indexes the result of an expression or function rather than the raw column - only a query that uses the same expression in its predicate (WHERE lower(email) = ...) can use it, which is what makes such predicates sargable.
Index Types (Hash, GIN, GiST, BRIN)	<pre>CREATE INDEX idx_status ON orders USING HASH (status) CREATE INDEX idx_tags ON docs USING GIN (tags)</pre>	- B-tree is the default; HASH stores a 32-bit hash and only helps = comparisons GIN builds an inverted index for arrays, JSONB or full-text search; GiST and BRIN suit geometric data and huge, naturally ordered tables.
DROP INDEX	<pre>DROP INDEX idx_last_name ON customers</pre>	Removes an index and frees its storage - SQL Server needs DROP INDEX name ON table ; PostgreSQL and MySQL use DROP INDEX name alone. Cannot drop an index backing a PRIMARY KEY or UNIQUE constraint.

TABLE 25

Views

Views package a SELECT statement under a name so it can be queried like a table. Standard views store no data: the underlying query re-runs on every reference, which is cheap on storage but offers no speed-up over the raw query. Materialized views flip that trade-off by persisting the result on disk for fast reads at the cost of staleness until the next refresh.

Concept	Example	Description
CREATE VIEW	<pre>CREATE VIEW active_customers AS SELECT * FROM customers WHERE active = 1</pre>	- Creates a named, saved query that can be queried like a table - not physically materialized: the defining query re-runs on every reference, so it costs no storage but gives no speed-up over the underlying SELECT.

Concept	Example	Description
CREATE OR REPLACE VIEW	<pre>CREATE OR REPLACE VIEW monthly_sales AS SELECT EXTRACT(MONTH FROM order_date) AS month, SUM(amount) FROM orders GROUP BY 1</pre>	• Replaces an existing view's query in place, keeping its permissions and dependent objects • the new query must keep the same column names, order and types; extra columns may only be appended at the end. SQL Server uses <code>ALTER VIEW</code> instead.
Updatable View	<pre>CREATE VIEW active_orders AS SELECT * FROM orders WHERE status = 'active'; UPDATE active_orders SET status = 'shipped' WHERE id = 42;</pre>	• A view whose <code>INSERT</code>, <code>UPDATE</code>, <code>DELETE</code> and <code>MERGE</code> statements pass straight through to the base table • a simple view (one table in <code>FROM</code>, no <code>DISTINCT</code>/<code>GROUP BY</code>/<code>HAVING</code>/<code>LIMIT</code>, no set operations or aggregates) qualifies automatically; anything more complex needs <code>INSTEAD OF</code> triggers.
WITH CHECK OPTION	<pre>CREATE VIEW eng_staff AS SELECT * FROM employees WHERE dept = 'Engineering' WITH CHECK OPTION</pre>	• Rejects <code>INSERT</code>, <code>UPDATE</code>, or <code>MERGE</code> through the view if the resulting row would no longer satisfy the view's <code>WHERE</code> clause • LOCAL checks only the current view; CASCADE (the default if neither is specified) also checks every underlying view.
Materialized View	<pre>CREATE MATERIALIZED VIEW report_summary AS SELECT dept, SUM(salary) FROM employees GROUP BY dept;</pre>	• Stores the query result physically on disk, so reads are fast but the data goes stale until the next refresh • PostgreSQL and Oracle; SQL Server's closest equivalent is an indexed view (below).
REFRESH MATERIALIZED VIEW (CONCURRENTLY)	<pre>REFRESH MATERIALIZED VIEW report_summary; REFRESH MATERIALIZED VIEW CONCURRENTLY report_summary;</pre>	• Recomputes a materialized view's stored data from its defining query, discarding the old contents • a plain refresh takes an exclusive lock; CONCURRENTLY avoids blocking readers but needs a <code>UNIQUE</code> index on the view covering all rows.
Indexed View (SQL Server)	<pre>CREATE VIEW dbo.vw_OrderTotals WITH SCHEMABINDING AS SELECT CustomerId, COUNT_BIG(*) AS Cnt FROM dbo.Orders GROUP BY CustomerId; CREATE UNIQUE CLUSTERED INDEX IX_vw_OrderTotals ON dbo.vw_OrderTotals(CustomerId);</pre>	• Materializes a view by building a unique clustered index on it, storing the result like a table • requires <code>WITH SCHEMABINDING</code>; the optimizer can then substitute the indexed view into other queries even without referencing it directly.
DROP VIEW	<pre>DROP VIEW IF EXISTS active_customers</pre>	• Removes only the view definition: underlying tables and their rows are unaffected • RESTRICT (the default) blocks the drop if other objects depend on the view; CASCADE drops them too.

TABLE 26

Transactions

Transactions group one or more SQL statements into an atomic unit so the database can guarantee ACID properties (Atomicity, Consistency, Isolation, Durability). The statements below mark transaction boundaries, set the visibility rules between concurrent sessions, and (in the case of `WITH (NOLOCK)`) trade correctness for raw read speed in SQL Server.

Concept	Example	Description
BEGIN TRANS ACTION	<pre>BEGIN TRANSACTION</pre>	• Starts an explicit transaction block: subsequent changes are held until <code>COMMIT</code> or <code>ROLLBACK</code> • <code>BEGIN / START TRANSACTION</code> in PostgreSQL and MySQL; most engines default to autocommit unless wrapped this way.
AUTOCOMMIT	<pre>\set AUTOCOMMIT off UPDATE accounts SET balance = balance - 100 WHERE id = 1; COMMIT;</pre>	• Default mode where each statement commits on its own the instant it succeeds, with no explicit transaction needed • turning it off means every statement runs inside an open transaction until an explicit <code>COMMIT</code> or <code>ROLLBACK</code> ends it.
COMMIT	<pre>COMMIT</pre>	• Permanently saves all changes made since the transaction started and frees the transaction's locks and resources • cannot be undone: once committed, <code>ROLLBACK</code> no longer applies.

Concept	Example	Description
ROLLBACK	<code>ROLLBACK</code>	- Discards all changes made since BEGIN (or since a named SAVEPOINT, with <code>ROLLBACK TO</code>) - used for error recovery and to abort transactions cleanly.
SAVEPOINT	<code>SAVEPOINT before_update</code> <code>ROLLBACK TO before_update</code>	- Creates a named checkpoint inside a transaction <code>ROLLBACK TO savepoint</code> undoes work back to that point without ending the transaction, so the outer transaction stays open and can still COMMIT.
SET TRANSACTION ISOLATION LEVEL	<code>SET TRANSACTION ISOLATION LEVEL READ COMMITTED</code>	- Controls which concurrency anomalies are allowed: dirty reads, non-repeatable reads, phantom reads - ascending strictness: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE (SQL Server also has SNAPSHOT); default is READ COMMITTED except MySQL InnoDB (REPEATABLE READ).
SELECT ... FOR UPDATE / FOR SHARE	<code>SELECT * FROM accounts WHERE id = 1 FOR UPDATE;</code> <code>-- row stays locked until COMMIT or ROLLBACK</code>	Locks the rows a query reads so other transactions can't change them first <code>FOR UPDATE</code> takes an exclusive lock (for a later write); <code>FOR SHARE</code> takes a shared lock; <code>NOWAIT</code> errors instead of waiting, <code>SKIP LOCKED</code> skips rows already locked.
Deadlock	<code>-- Session A:</code> <code>UPDATE accounts SET balance = balance - 100 WHERE acctnum = 11111;</code> <code>-- Session B (blocked):</code> <code>UPDATE accounts SET balance = balance - 100 WHERE acctnum = 22222;</code> <code>UPDATE accounts SET balance = balance + 100 WHERE acctnum = 11111;</code>	- Two transactions each hold a lock the other needs, so neither can proceed - the database detects the cycle and aborts one transaction automatically; avoid it by acquiring locks on objects in the same order everywhere.
WITH (NOLOCK)	<code>SELECT * FROM orders WITH (NOLOCK)</code>	- SQL Server table hint equivalent to READ UNCOMMITTED for that table: reads without acquiring shared locks - can return uncommitted (dirty) data, duplicate rows, or miss committed rows; a notorious "go-fast" anti-pattern.

TABLE 27

Access Control

Data Control Language governs who can do what: granting and revoking privileges on objects, creating the roles or users those privileges attach to, and restricting which rows a role can even see.

Concept	Example	Description
GRANT	<code>GRANT SELECT, INSERT ON accounts TO joe;</code>	Gives a role one or more privileges on an object
REVOKE	<code>REVOKE ALL ON accounts FROM PUBLIC;</code>	Removes a previously granted privilege
CREATE ROLE	<code>CREATE ROLE joe LOGIN PASSWORD 'pw123';</code>	- Defines a new database role <code>NOLOGIN</code> by default, so it cannot itself connect unless <code>LOGIN</code> is given
CREATE USER	<code>CREATE USER joe PASSWORD 'pw123';</code>	Alias for <code>CREATE ROLE</code> with <code>LOGIN</code> assumed instead of <code>NOLOGIN</code>
WITH GRANT OPTION	<code>GRANT SELECT ON accounts TO joe WITH GRANT OPTION;</code>	- Lets the grantee re-grant the privilege to others - Revoking the grant option cascades to everyone who got it through that chain
Role membership	<code>GRANT managers TO joe;</code>	Adds a role as a member of another role, inheriting its privileges
DENY	<code>DENY SELECT ON dbo.accounts TO joe;</code>	SQL Server statement that blocks a permission, overriding any <code>GRANT</code> except for object owners or sysadmin

Concept	Example	Description
ALTER DEFAULT PRIVILEGES	<pre>ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO joe;</pre>	Sets privileges applied automatically to objects created later, not to existing ones
Row-Level Security	<pre>ALTER TABLE accounts ENABLE ROW LEVEL SECURITY; CREATE POLICY account_managers ON accounts TO managers USING (manager = current_user);</pre>	- Restricts which rows a role can see or modify with a policy expression - No policy plus RLS enabled means no rows are visible

TABLE 28

Error Handling

SQL Server's structured error handling centers on **TRY...CATCH** : any runtime error of severity above 10 inside the TRY block jumps execution to the matching CATCH block, where a family of **ERROR_*** functions exposes the message, number, severity, line, and procedure of the original error. Inside CATCH you also check **XACT_STATE()** to decide whether to commit or roll back, and re-surface errors with **THROW** (preferred since SQL Server 2012) or the older **RAISERROR** .

Construct	Example	Description
TRY...CATCH	<pre>BEGIN TRY INSERT INTO orders VALUES (1, 99.99) END TRY BEGIN CATCH SELECT ERROR_MESSAGE(), ERROR_NUMBER() END CATCH</pre>	Structured error handling (SQL Server / Azure SQL) - any runtime error of severity > 10 in TRY transfers control to CATCH; compile and name-resolution errors at the same scope are NOT caught.
ERROR_MESSAGE()	<pre>SELECT ERROR_MESSAGE()</pre>	- Returns the error message text of the error that triggered the CATCH block - returns NULL outside CATCH
ERROR_NUMBER()	<pre>SELECT ERROR_NUMBER()</pre>	- Returns the error number, which correlates to <code>sys.messages</code> for system errors - user-defined errors use 50000+ - unlike <code>@@ERROR</code> , the value persists across the whole CATCH scope.
ERROR_SEVERITY() / ERROR_LINE() / ERROR_PROCEDURE()	<pre>SELECT ERROR_SEVERITY(), ERROR_LINE(), ERROR_PROCEDURE()</pre>	- Return the severity, line number, and procedure name of the error that invoked the CATCH block - each returns NULL outside CATCH; <code>ERROR_PROCEDURE()</code> also returns NULL with no procedure or trigger involved.
XACT_STATE()	<pre>IF XACT_STATE() = -1 ROLLBACK IF XACT_STATE() = 1 COMMIT</pre>	- Returns 1 (active committable), -1 (active uncommittable, must rollback), or 0 (no active transaction) - essential check in CATCH blocks before deciding to COMMIT or ROLLBACK.
THROW	<pre>THROW 50001, 'Invalid customer ID', 1 -- or to re-throw: THROW</pre>	- Raises a user-defined error (number ≥ 50000, severity always 16) or re-throws the original error with its original details (bare THROW in CATCH) - SQL Server 2012+; preferred over <code>RAISERROR</code> for new code; honors <code>SET XACT_ABORT</code> .
RAISERROR()	<pre>RAISERROR('Custom error: %s', 16, 1, @detail)</pre>	Older mechanism for raising errors; supports custom severity 1-25 and printf-style message formatting (<code>%s</code> , <code>%d</code>) - does not honor <code>SET XACT_ABORT</code> ; still useful when you need a custom severity.

TABLE 29

JSON Functions

SQL Server exposes JSON through a dedicated function family (`JSON_VALUE` , `JSON_QUERY` , `OPENJSON`) on plain `nvarchar` or its native `json` type, while PostgreSQL, MySQL and Oracle share the SQL standard's SQL/JSON functions (`JSON_EXISTS` , `JSON_TABLE` , `IS JSON`) alongside PostgreSQL's own `->` / `->>` / `@>` operators. Scalars use `JSON_VALUE` , objects and arrays use `JSON_QUERY` , and rowset shredding uses `OPENJSON` or `JSON_TABLE` ; most path-based functions accept a `lax` (silent NULL) or `strict` (raise error) mode.

Function	Example	Description
JSON_VALUE()	<code>JSON_VALUE(payload, '\$.customer.name')</code>	- Extracts a scalar value (string, number, boolean) from a JSON path - returns NULL in lax mode (default) if the path is missing or points to an object/array.
-> / ->>	<pre>'{"a":{"b":"foo"}}'::jsonb -> 'a' → {"b":"foo"} '{"a":1,"b":2}'::jsonb ->> 'b' → 2</pre>	-> extracts a JSON object field or array element as JSON ->> extracts the same value as text; both also work in MySQL.
JSON_QUERY()	<code>JSON_QUERY(payload, '\$.address')</code>	- Extracts a JSON object or array fragment - returns NULL for scalar values in lax mode (the opposite of <code>JSON_VALUE</code>).
ISJSON()	<code>WHERE ISJSON(payload) = 1</code>	- Returns 1 if the string is valid JSON, 0 if not, NULL if input is NULL 2022+ accepts a type constraint: <code>VALUE</code>, <code>ARRAY</code>, <code>OBJECT</code>, <code>SCALAR</code>.
IS JSON	<code>SELECT '{"a": "b"}'::jsonb IS JSON OBJECT;</code> <code>→ t</code>	- SQL standard predicate that tests whether a value can be parsed as JSON - narrows to <code>VALUE</code> , <code>SCALAR</code> , <code>ARRAY</code> or <code>OBJECT</code> ; used in PostgreSQL 17+ and Oracle (SQL Server uses <code>ISJSON()</code> instead).
OPENJSON()	<code>SELECT * FROM OPENJSON(@json) WITH (id INT, name VARCHAR(50))</code>	Parses JSON into a rowset (SQL Server 2016+) - without <code>WITH</code> returns generic key/value/type columns; <code>WITH</code> defines typed columns and supports <code>AS JSON</code> for nested fragments.
JSON_TABLE()	<code>SELECT jt.* FROM my_films, JSON_TABLE(js, '\$.favorites[*]' COLUMNS (kind text PATH '\$.kind', title text PATH '\$.films[*].title'));</code>	- The SQL standard's rowset shredder, the PostgreSQL/MySQL/Oracle equivalent of <code>OPENJSON</code> - the <code>COLUMNS</code> clause maps path expressions to typed output columns.
JSON_EXISTS()	<code>JSON_EXISTS(jsonb '{"key1": [1,2,3]}' , 'strict \$.key1[*] ? (@ > 2)')</code> <code>→ t</code>	- Returns true if a path expression yields any item, false otherwise - standard SQL/JSON function, also in PostgreSQL 17+ and Oracle.
@>	<code>'{"a":1, "b":2}'::jsonb @> '{"b":2}'::jsonb</code> <code>→ t</code>	- Tests whether one jsonb value contains another (PostgreSQL) - the reverse test is <code><@</code> ; often paired with a GIN index for fast lookups.
JSON_OBJECT()	<code>JSON_OBJECT('id':id, 'name':name)</code>	Constructs a JSON object from key:value pairs (SQL Server 2022+) - default NULL ON NULL keeps NULL keys; ABSENT ON NULL omits them.
JSON_ARRAY()	<code>JSON_ARRAY(1, 'two', NULL)</code>	Constructs a JSON array from values (SQL Server 2022+) - default ABSENT ON NULL silently drops NULL elements (opposite of <code>JSON_OBJECT</code> default).
JSON_ARRAYAGG() / JSON_OBJECTAGG()	<code>SELECT json_arrayagg(v) FROM (VALUES(2),(1)) t(v);</code> <code>→ [2, 1]</code>	Aggregate forms of <code>JSON_ARRAY</code> / <code>JSON_OBJECT</code>, collecting a group of rows into one JSON value - standard SQL/JSON functions, also in MySQL, Oracle and SQL Server 2025.
JSON_MODIFY()	<code>JSON_MODIFY(payload, '\$.status', 'active')</code>	- Returns a new JSON string with one property updated - setting NULL in lax mode deletes the key - use the <code>append</code> keyword to push onto an array.

Function	Example	Description
FOR JSON PATH	<pre>SELECT id, firstName AS "info.name" FROM People FOR JSON PATH;</pre>	- Formats an entire query result set as a JSON array (SQL Server) - dot-separated column aliases build nested objects; FOR JSON AUTO infers the shape automatically instead.

References

Official Documentation

1. SELECT
2. SELECT * vs explicit column list
3. FROM
4. VALUES lists as a table
5. WHERE
6. ORDER BY
7. OFFSET
8. 6. QUALIFY
9. 8. ORDER BY
10. Comparison operators
11. AND / OR / NOT
12. IN
13. ANY / SOME
14. CROSS APPLY / OUTER APPLY
15. COUNT()
16. SUM()
17. AVG()
18. MAX()
19. MIN()
20. GROUP_CONCAT / STRING_AGG
21. FILTER (WHERE ...)
22. VARIANCE / VAR_SAMP
23. STDDEV / STDEV
24. APPROX_COUNT_DISTINCT()
25. PRODUCT()
26. Three-Valued Logic (UNKNOWN)
27. COALESCE()
28. ISNULL() / IFNULL() / NVL()
29. Simple CASE
30. IIF()
31. CHOOSE()
32. DECODE()
33. ORDER BY column position
34. TOP WITH TIES
35. CTE (Common Table Expression)
36. UNION
37. INTERSECT
38. ROW_NUMBER()
39. RANK()
40. DENSE_RANK()
41. NTILE()
42. PERCENT_RANK()
43. CUME_DIST()
44. SUM() OVER()
45. AVG() OVER()
46. COUNT() OVER()
47. LAG()
48. LEAD()
49. FIRST_VALUE()
50. LAST_VALUE()
51. NTH_VALUE()
52. N PRECEDING / N FOLLOWING
53. GROUPING()
54. GROUPING_ID()
55. PIVOT
56. crosstab()
57. CONCAT()
58. Concatenation Operator (||)
59. CONCAT_WS()
60. TRIM() / LTRIM() / RTRIM()
61. LEFT() / RIGHT()
62. LEN() / LENGTH()
63. REPLACE()
64. CHARINDEX() / INSTR() / POSITION()
65. STRING_SPLIT()
66. FORMAT()
67. LIKE
68. ILIKE
69. REGEXP_LIKE / SIMILAR TO / ~ (tilde)
70. REGEXP_REPLACE
71. REGEXP_SUBSTR / REGEXP_INSTR
72. REGEXP_COUNT
73. CONTAINS
74. GETDATE() / NOW() / CURRENT_TIMESTAMP
75. CURRENT_DATE
76. DATEADD() / DATE_ADD() / INTERVAL
77. DATEDIFF()
78. DATEPART() / EXTRACT()
79. AT TIME ZONE
80. CAST() (to date/time)
81. DATEFROMPARTS() / MAKE_DATE()
82. TO_CHAR() / FORMAT()
83. EOMONTH() / LAST_DAY()
84. DATE_BUCKET()
85. ROUND()
86. FLOOR() / CEILING()
87. ABS()
88. LOG() / LOG10() / LN()
89. SIGN()
90. RAND() / RANDOM()
91. Integer Types
92. Character Types
93. Date/Time Types
94. Boolean
95. UUID
96. JSON / JSONB
97. Binary Types: BYTEA / VARBINARY
98. TRY_CAST()
99. TRY_CONVERT()
100. Implicit Conversion

101. `PARSE()`
103. `STR()`
105. `CREATE TABLE AS SELECT`
107. `Generated Column`
109. `ALTER TABLE MODIFY / ALTER COLUMN`
111. `TRUNCATE TABLE`
113. `CREATE SCHEMA`
115. `PRIMARY KEY`
117. `NOT NULL`
119. `DEFAULT`
121. `UPDATE ... SET`
123. `DELETE FROM`
125. `RETURNING / OUTPUT clause`
127. `EXPLAIN / EXPLAIN ANALYZE`
129. `Covering Index (INCLUDE)`
131. `Filtered Index (Partial Index)`
133. `Index Types (Hash, GIN, GiST, BRIN)`
135. `CREATE VIEW`
137. `REFRESH MATERIALIZED VIEW (CONCURRENTLY)`
139. `DROP VIEW`
141. `AUTOCOMMIT`
143. `ROLLBACK`
145. `SET TRANSACTION ISOLATION LEVEL`
147. `WITH (NOLOCK)`
149. `REVOKE`
151. `CREATE USER`
153. `DENY`
155. `Row-Level Security`
157. `ERROR_MESSAGE()`
159. `ERROR_SEVERITY() / ERROR_LINE() / ERROR_PROCEDURE()`
161. `THROW`
163. `JSON_VALUE()`
165. `JSON_QUERY()`
167. `OPENJSON()`
169. `JSON_ARRAY()`
171. `FOR JSON PATH`
173. `REPEAT() / REPLICATE()`
175. `REGEXP_LIKE / SIMILAR TO / ~ (tilde)`
177. `EOMONTH() / LAST_DAY()`
179. `Date/Time Types`
181. `UUID`
183. `Binary Types: BYTEA / VARBINARY`
185. `Identity Column`
187. `RETURNING / OUTPUT clause`
189. `ERROR_LINE()`
191. `ISNULL() / IFNULL() / NVL()` - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/coalesce-transact-sql?view=sql-server-ver17>

Articles & Tutorials

193. `Correlated Subquery`
195. `Composite Index`
197. `Implicit Conversion`
199. `DELETE ... USING (DELETE with JOIN)`

Encyclopedic

200. `Sargable Predicate`

102. `TRY_PARSE()`
104. `CREATE TABLE`
106. `Identity Column`
108. `ALTER TABLE ADD COLUMN`
110. `DROP TABLE`
112. `CREATE SEQUENCE`
114. `CREATE DATABASE`
116. `FOREIGN KEY`
118. `UNIQUE`
120. `INSERT INTO ... VALUES`
122. `UPDATE with JOIN`
124. `MERGE (UPSERT)`
126. `CREATE INDEX`
128. `CREATE UNIQUE INDEX`
130. `Clustered Index`
132. `Expression Index (Function-Based Index)`
134. `DROP INDEX`
136. `Materialized View`
138. `Indexed View (SQL Server)`
140. `BEGIN TRANSACTION`
142. `COMMIT`
144. `SAVEPOINT`
146. `Deadlock`
148. `GRANT`
150. `CREATE ROLE`
152. `WITH GRANT OPTION`
154. `ALTER DEFAULT PRIVILEGES`
156. `TRY...CATCH`
158. `ERROR_NUMBER()`
160. `XACT_STATE()`
162. `RAISERROR()`
164. `-> / ->>`
166. `ISJSON()`
168. `JSON_OBJECT()`
170. `JSON_MODIFY()`
172. `Concatenation Operator (||)`
174. `TRANSLATE()`
176. `REGEXP_SUBSTR / REGEXP_INSTR`
178. `Character Types`
180. `Boolean`
182. `UUID`
184. `Identity Column`
186. `DELETE ... USING (DELETE with JOIN)`
188. `Row-Level Security`
190. `ERROR_PROCEDURE()`
192. `POWER() / SQRT()` - <https://learn.microsoft.com/en-us/sql/t-sql/functions/power-transact-sql>

194. `Conditional Aggregation`
196. `FILTER (WHERE ...)`
198. `INSERT ... ON CONFLICT (UPSERT)`